InfoGuard — Project Briefing for AI Handoff

> Paste this entire document at the start of a new chat.
> The new AI should read it and continue seamlessly from Phase 2.

---

1. Project Overview

Project name: InfoGuard **Type:** Student-led academic project — working prototype + governmental proposal **Core idea:** Combine the graph-theory-based misinformation suppression algorithm from an academic paper with a propagation-aware GNN classifier to build a system that counters misinformation without explicit content banning.

The paper this is based on:

> Bayiz & Topcu (2022). "Countering Misinformation on Social Networks Using Graph Alterations." arXiv:2211.04617v1

The paper's algorithm in one paragraph: Model a social network as a Stochastic Block Model (SBM) with k polarization partitions. True and false content propagate according to two different SBMs: G^+ (true) and G^- (false), with edge probability matrices b^+_uv and b^-uv. *Run a discrete-time SIR cascade on these networks. At each cascade step t, solve a Linear Program (Algorithm 2) that finds dropout probabilities d*uv — the probability of randomly hiding content in a user's feed for each pair of polarization classes u,v. The LP minimises E[|I{t+1}|] for false content while keeping E[|I*{t+1}|] ⩾ α|I_t| for true content (α is the safety parameter, default 1.5). Apply these dropouts to the real network without ever identifying or banning individual pieces of content.

The project's original contribution on top of the paper: The paper assumes b^+ and b^- are known (fitted from labeled data). This project adds a **propagation-aware GNN (BiGCN)** as the classifier that produces those labels automatically, replacing the need for manual content classification. This closes the loop: BiGCN labels content → labels feed SBM fitting → LP optimizer applies dropouts.

Use case framing: A governmental emergency communications system for wartime or disaster scenarios. The dashboard shows network operators the live cascade state, dropout heatmaps, and content alerts without requiring real-time content banning.

---

2. Technology Decisions Made (and Why)

Classifier: BiGCN (not Claude API, not RoBERTa)

- **Decision:** Use BiGCN (Bi-directional Graph Convolutional Network) as the primary classifier.
- **Why:** BiGCN classifies content by how it propagates through the network, not just by text. This directly aligns with the paper's philosophy — the same structural signal the LP optimizer exploits is what BiGCN learns to detect. It runs locally (no external API), is free to call, and has published benchmarks on Twitter15/16.
- **Rejected options:**
  - Claude API alone: too slow, expensive per-call, requires external server — unsuitable for government wartime infrastructure.
  - RoBERTa fine-tuned on LIAR: text-only, domain gap for crisis scenarios, misses structural signal.
  - GCAN: considered, BiGCN chosen because it's simpler and has cleaner open-source implementation.

Datasets

- **Twitter15 / Twitter16:** Used exclusively for training and evaluating BiGCN. These have rich propagation trees (depth 3–12, genuine retweet chains). Source: https://github.com/TianBian95/BiGCN (rumdetect2017.zip via Dropbox). NOT used for the main InfoGuard pipeline.
- **WICO Graph + WICO Text:** Primary dataset for fitting SBM matrices and running the full InfoGuard pipeline (Phase 4). Matches the paper exactly — it is the dataset used in the paper's Table II. Source: https://datasets.simula.no/wico-graph and https://datasets.simula.no/wico-text
- **WICO structure:**
  - WICO Text: three .txt files (5g_corona_conspiracy.txt, other_conspiracy.txt, non_conspiracy.txt), one bare tweet ID per line
  - WICO Graph: three subfolders (5G_Conspiracy_Graphs/, Other_Graphs/, Non_Conspiracy_Graphs/), each containing numbered subdirectories (1, 2, 3...) — NOT tweet IDs. Each subdirectory has edges.txt (space-separated node pairs) and nodes.csv (id, time, friends, followers).
  - **Critical:** WICO Graph folder names are sequential integers, NOT tweet IDs. Joining on tweet ID fails silently. Labels must be assigned from folder name only.
- **PHEME:** Mentioned as optional, not yet used.
- **FakeNewsNet:** Mentioned as optional, not yet used.

Graph theory engine: scipy.optimize.linprog

- LP from the paper implemented via scipy. No other solver needed for the prototype.

Language and stack

- Python 3.11
- PyTorch 2.1 (CPU is fine for prototyping)
- PyTorch Geometric (PyG) 2.4 — install AFTER torch
- NetworkX 3.2
- HuggingFace transformers (RoBERTa text encoder inside BiGCN)
- Flask (Phase 5 dashboard API)
- Jupyter notebooks for Phase 1 exploration

---

3. Project Folder Structure

```
infoshield/
├── config.py                          ← CENTRAL CONFIG — import cfg everywhere
├── requirements.txt
├── data/
│   ├── raw/
│   │   ├── wico-text/                 ← WICO Text files
│   │   ├── wico-graph/                ← WICO Graph folders
│   │   ├── twitter15/
│   │   │   ├── tree/                  ← one .txt per event
│   │   │   ├── label.txt              ← format: "label:tweet_id"
│   │   │   └── source_tweets.txt      ← format: "tweet_id\ttext"
│   │   └── twitter16/                 ← identical structure to twitter15
│   └── processed/
│       ├── graphs/                    ← PyG Data objects (.pt)
│       └── sbm_matrices/              ← fitted b+, b- numpy arrays
├── gnn/
│   ├── dataset.py                     ← PyG dataset loader (TO BUILD)
│   ├── bigcn.py                       ← BiGCN architecture (TO BUILD)
│   ├── train.py                       ← training loop (TO BUILD)
│   ├── evaluate.py                    ← accuracy, F1 (TO BUILD)
│   └── predict.py                     ← inference on new events (TO BUILD)
├── graph_engine/
│   ├── network_model.py               ← SBM construction (TO BUILD)
│   ├── sir_simulation.py              ← cascade simulator (TO BUILD)
│   └── optimizer.py                   ← LP Algorithm 2 (TO BUILD)
├── pipeline/
│   ├── sbm_fitter.py                  ← b+/b- estimation (TO BUILD)
│   └── run_pipeline.py                ← end-to-end runner (TO BUILD)
├── api/
│   └── server.py                      ← Flask REST API (TO BUILD)
├── dashboard/
│   └── index.html                     ← government operator UI (TO BUILD)
├── evaluation/
│   ├── propagation_tree_summary.csv   ← GENERATED by explore_data.ipynb ✓
│   ├── sample_tree_ids.csv            ← GENERATED ✓
│   ├── contrast_tree_ids.csv          ← GENERATED ✓
│   └── tree_figures/                  ← all Phase 1 plots ✓
└── explore_data.ipynb                 ← COMPLETE ✓
    visualize_trees.ipynb              ← COMPLETE ✓
```

---

4. config.py — Key Values

The central config object is `cfg`. Import it everywhere with `from config import cfg`.

Key values relevant to Phase 2+:

```python
cfg.paths.twitter15        # Path to twitter15 raw data
cfg.paths.twitter15_trees  # Path to twitter15/tree/
cfg.paths.twitter16        # Path to twitter16 raw data
cfg.paths.twitter16_trees  # Path to twitter16/tree/
cfg.paths.wico_graph       # Path to wico-graph/
cfg.paths.wico_text        # Path to wico-text/
cfg.paths.graphs_pt        # Where to save PyG .pt files
cfg.paths.sbm_matrices     # Where to save fitted SBM matrices

cfg.twitter15.label_file          # "label.txt"
cfg.twitter15.source_tweets_file  # "source_tweets.txt"
cfg.twitter15.tree_dir            # "tree"
cfg.twitter15.root_sentinel       # "ROOT"
cfg.twitter15.min_tree_size       # 3
cfg.twitter15.max_tree_size       # 2000
cfg.twitter15.binary_label_map    # {0:"true", 1:"false", 2:"uncertain", 3:"true"}
# twitter16 has identical config

cfg.wico.graph_dirs        # {0:"5G_Conspiracy_Graphs", 1:"Other_Graphs", 2:"Non_Con
cfg.wico.graph_edges_file  # "edges.txt"
cfg.wico.graph_nodes_file  # "nodes.csv"
cfg.wico.binary_label_map  # {0:"false", 1:"false", 2:"true"}

cfg.bigcn.text_encoder     # "roberta-base"
cfg.bigcn.text_embed_dim   # 768
cfg.bigcn.freeze_encoder   # True (freeze RoBERTa weights during training)
cfg.bigcn.gcn_hidden_dim   # 256
cfg.bigcn.gcn_output_dim   # 128
cfg.bigcn.gcn_num_layers   # 2
cfg.bigcn.dropout          # 0.3
cfg.bigcn.num_epochs       # 200
cfg.bigcn.batch_size       # 32
cfg.bigcn.learning_rate    # 5e-4
cfg.bigcn.patience         # 20 (early stopping)
cfg.bigcn.num_classes      # 4 (Twitter15: true/false/unverified/non-rumor)

cfg.lp.alpha               # 1.5 (LP safety parameter)
cfg.lp.lambda_weight       # 1.0 (softened LP weight)
cfg.sbm.num_partitions     # 13 (matching the paper's WICO analysis)
cfg.sbm.clustering_resolution  # 2.0 (modularity clustering resolution)
cfg.sbm.label_confidence_threshold  # 0.65
```

---

5. Twitter15/16 Tree File Format

Each tree file is at `tree/<tweet_id>.txt`. One edge per line:

```
['parent_uid', 'parent_tweet_id', 'delay_min']->['child_uid', 'child_tweet_id',
'delay_min']
```

Root sentinel line (source tweet has no real parent):

```
['ROOT', 'ROOT', '0.0']->['972651', '80080680482123777', '0.0']
```

All delay values are floats (minutes since source tweet).

`label.txt` format: `label:tweet_id` (e.g. `false:656955120626880512`) `source_tweets.txt` format: `tweet_id\ttext` (tab-separated)

4-class labels: true, false, unverified, non-rumor **Binary mapping:** true→true, false→false, unverified→uncertain, non-rumor→true

---

6. Phase 1 — COMPLETE ✅

What was done

- Built `explore_data.ipynb`: loads Twitter15, Twitter16, WICO Text, WICO Graph; computes tree statistics for all 4,765 cascades; saves `propagation_tree_summary.csv`.
- Built `visualize_trees.ipynb`: produces all Phase 1 figures.
- Fixed 6 bugs across both notebooks (WICO label assignment, `make_all()` directory bug, `dataset_cfg` parameter, double-plot bug, `squeeze=False` axes bug, branching ratio formula).

Key empirical findings confirmed

1. **False content spreads wide and shallow:** best false example — 1748 nodes, max depth 3, 1583 nodes at depth 1.
2. **True content spreads deep and narrow:** best true example — 642 nodes, max depth 12, gradual decay across all levels.
3. **Branching ratio (geometric mean of per-step ratios):** false=1.9, true=1.0. ~35% of Twitter15/16 cascades exceed α=1.5, confirming LP feasibility.
4. **WICO class balance:** 1905 true (non-conspiracy), 597 false (5G + other conspiracy) — 76%/24% imbalance relevant for Phase 4.

Generated files (all in evaluation/)

- `propagation_tree_summary.csv` — 4,765 rows, columns: dataset, tweet_id, content_id, root_user, label, raw_label, graph_path, text, num_nodes, num_edges, cascade_size, max_depth, max_width, branching_ratio, num_roots, is_arborescence, temporal_span
- `sample_tree_ids.csv` — 6 selected WICO examples
- `contrast_tree_ids.csv` — best structural contrasts (deepest true + widest false)
- `tree_figures/showcase_false_vs_true_spreading.png` — main hero figure
- `tree_figures/depth_profiles_false_vs_true.png` — strongest individual figure
- `tree_figures/stats_comparison_false_vs_true.png`
- `tree_figures/aggregate_distributions_false_vs_true.png`
- `tree_figures/bigcn_dual_view.png`

Known remaining limitation (acceptable)

BiGCN dual view shows solid color triangles because 300+ nodes overlap at the same y-coordinate. Not a code bug — a fundamental rendering limitation with star-shaped trees. Accepted as-is.

---

7. What Still Needs to Be Done

Phase 2 — BiGCN Classifier (NEXT STEP)

Build files in `gnn/`:

`gnn/dataset.py`

- Load Twitter15/16 tree files from `cfg.paths.twitter15_trees` and `cfg.paths.twitter16_trees`
- Parse each tree file using the edge format above
- For each tree, create a PyG `Data` object with:
  - `x`: node feature matrix — RoBERTa embedding of root tweet text (768-dim) for root node; zero vector or degree features for other nodes
  - `edge_index`: COO format adjacency (top-down direction)
  - `edge_index_bu`: reversed edge_index (bottom-up direction for BU-GCN branch)
  - `y`: label (integer from `cfg.twitter15.label_map`)
  - `root_id`: the root user ID
- Save processed graphs as `.pt` files to `cfg.paths.graphs_pt`
- Implement train/val/test split (standard for Twitter15: 5-fold cross-validation)

`gnn/bigcn.py` BiGCN architecture (from paper: Bian et al. 2020 "Rumor Detection on Social Media with Bi-Directional Graph Convolutional Networks"):

- Two GCN branches: TD-GCN (top-down) and BU-GCN (bottom-up)
- Each branch: `num_layers` GCN layers with hidden_dim=256, output_dim=128
- Root node feature (RoBERTa embedding) is used as the initial feature for all nodes (broadcast from root)
- TD and BU embeddings are concatenated → 256-dim
- Linear classifier head → num_classes (4 for Twitter15)
- Dropout=0.3 between layers
- Note: `cfg.bigcn.freeze_encoder=True` means RoBERTa is called offline to generate embeddings, stored in the dataset, NOT inside the BiGCN forward pass

`gnn/train.py`

- Standard PyG training loop
- Adam optimizer, lr=5e-4, weight_decay=1e-4
- CrossEntropyLoss
- Early stopping with patience=20
- Save best checkpoint to `cfg.paths.bigcn_checkpoint`
- Log: epoch, train_loss, val_loss, val_accuracy, val_F1

`gnn/evaluate.py`

- Load best checkpoint
- Compute accuracy, macro F1, confusion matrix on test set
- Target benchmarks (from BiGCN paper on Twitter15): ~88% accuracy 4-class
- Also compute binary (true vs false) metrics since that's what feeds Phase 3

`gnn/predict.py`

- Load trained model
- Accept a new propagation tree (as a graph_path or DiGraph)
- Return: label (true/false/uncertain), confidence score (0–1)
- This output feeds `pipeline/sbm_fitter.py` in Phase 3

Phase 3 — Graph Theory Engine

Build files in `graph_engine/`. Implements Algorithm 2 from the paper directly.

`graph_engine/network_model.py`

- `SBM` class: holds b^+ and b^- matrices (k×k numpy arrays)
- Constructor takes polarization partition assignments from WICO Graph modularity clustering

`graph_engine/sir_simulation.py`

- Simulate discrete-time SIR cascade (infectious period m=1, equivalent to independent cascade)
- Given G (real network), dropout matrix d*, S_t, I_t, R_t → returns S_{t+1}, I_{t+1}, R_{t+1}

`graph_engine/optimizer.py`

- Implement Algorithm 2 exactly as in the paper
- Primary LP (eq. 22): `scipy.optimize.linprog` minimizing false content spread subject to true content branching constraint
- Softened LP (eq. 24): used when primary is infeasible (low-virality content)
- Feasibility check (eq. 23) to decide which LP to run
- Returns d* (k×k dropout probability matrix)

Phase 4 — Integration Pipeline

`pipeline/sbm_fitter.py`

- Take BiGCN-labeled WICO cascades (confidence ⩾ cfg.sbm.label_confidence_threshold)
- Run modularity clustering (resolution=2.0) on union of all WICO graphs → 13 polarization classes
- Estimate b^+_uv and b^-_uv by frequentist counting of content transfers between polarization classes
- Save to `cfg.paths.sbm_matrices`

`pipeline/run_pipeline.py`

- End-to-end: load WICO data → BiGCN classify → fit SBMs → run Algorithm 2 per cascade step → apply dropouts → measure R∞ reduction
- Reproduce Table II from the paper: test (α,λ) = (1.5,1), (2,1.5), (3,2) vs control
- Target: ~45% cascade size reduction for false content, branching ratio ⩾ 1.5 for true

Phase 5 — Government Dashboard

`api/server.py` — Flask REST endpoints:

- POST `/classify` — run BiGCN on a new content item
- POST `/simulate` — simulate cascade with dropout
- GET `/dropout_matrix` — return current d* as JSON
- GET `/network_state` — return S_t, I_t, R_t sizes

`dashboard/index.html` — Operator UI:

- Network graph visualization (D3.js or vis.js) showing polarization clusters
- Cascade simulation panel
- k×k dropout probability heatmap
- Alert feed for uncertain/high-confidence false items

---

8. Important Technical Notes for Phase 2

1. **Install order matters for PyG:**

```bash
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install torch-geometric
pip install torch-scatter torch-sparse -f https://data.pyg.org/whl/torch-2.1.0+cp
pip install -r requirements.txt
```

2. **RoBERTa embeddings are pre-computed offline.** Do not put the RoBERTa model inside BiGCN's forward pass — it would be called once per batch forward which is very slow. Instead, compute root tweet embeddings once during dataset preprocessing and store them as node features in the PyG Data objects.
3. **Twitter15 uses 5-fold cross-validation** (standard in all BiGCN/GCAN papers). Do not use a simple 80/10/10 split or your results won't be comparable to published benchmarks.
4. **The binary label mapping for the SBM pipeline:** When BiGCN predicts a 4-class label, convert to binary using `cfg.twitter15.binary_label_map`: true→"true", false→"false", unverified→"uncertain", non-rumor→"true". Only high-confidence "true" and "false" predictions (confidence ⩾ 0.65) feed the SBM fitter.
5. **Geometric-mean branching ratio** (already implemented in explore_data.ipynb, repeat in any new code):

```python
step_ratios = [depth_counts[d+1]/depth_counts[d]
               for d in sorted(depth_counts) if d+1 in depth_counts]
log_ratios = [np.log(r) for r in step_ratios if r > 0]
branching_ratio = float(np.exp(np.mean(log_ratios))) if log_ratios else 0.0
```

6. **WICO Graph root node heuristic:** WICO edges.txt is a follower network, not a temporal retweet chain. The "root" (source tweet author) is identified as the node with the highest follower count in nodes.csv, not by in-degree=0.
7. **cfg.paths.make_all() bug:** The function tries to mkdir on `bigcn_checkpoint` which is a file path, creating a spurious directory. Do NOT call `cfg.paths.make_all()`. Create directories manually using the explicit list in explore_data.ipynb Cell 03.

---

9. Evaluation Targets (from the Paper)

When the full pipeline runs on WICO, the expected results from Table II are:

α	λ	E[R∞] false	E[R∞] true	P[R∞<5] false	P[R∞<5] true
control	—	48.7	50.1	0.00	0.01
1.5	1	26.1	32.8	0.13	0.05
2	1.5	28.2	39.4	0.09	0.04
3	2	36.0	41.1	0.02	0.00

The project should reproduce these numbers in Phase 4's `evaluation/results.ipynb`.

---

10. Files That Currently Exist (Written and Tested)

File	Status
`config.py`	✅ Complete
`requirements.txt`	✅ Complete
`explore_data.ipynb`	✅ Complete, runs cleanly
`visualize_trees.ipynb`	✅ Complete, runs cleanly
`classifier/ai_classifier.py`	✅ Written (Claude API version, superseded by BiGCN decision but kept as fallback)
All `gnn/` files	❌ Not yet built
All `graph_engine/` files	❌ Not yet built
All `pipeline/` files	❌ Not yet built
All `api/` and `dashboard/` files	❌ Not yet built

---

11. How to Pick Up

You are starting Phase 2. The first file to build is `gnn/dataset.py`.

Ask the student to confirm:

1. That `explore_data.ipynb` has been run and `evaluation/propagation_tree_summary.csv` exists.
2. That Twitter15 and Twitter16 datasets are present at `cfg.paths.twitter15` and `cfg.paths.twitter16` with the `tree/` subdirectory.
3. Whether they want to start with the PyG dataset loader (`gnn/dataset.py`) or the model architecture (`gnn/bigcn.py`) first. Recommended order: dataset.py → bigcn.py → train.py → evaluate.py → predict.py.

The build order for Phase 2 is:

```
gnn/dataset.py → gnn/bigcn.py → gnn/train.py → gnn/evaluate.py → gnn/predict.py
```

Phase 3 (graph_engine/) can be built in parallel with Phase 2 since the two modules are independent until Phase 4.